

AI OPERATIONS — MODULE 3 • LECTURE 1

Docker & Containerization for AI

A self-contained ~3-hour lecture + hands-on session: Images & Containers • Dockerfiles • Multi-Stage Builds • GPU Passthrough

Agenda — ~3 Hours

Time	Block	Format
0:00 – 0:15	Hook: you followed Module 1 perfectly, it still doesn't reproduce	Interactive lecture
0:15 – 0:30	Images vs. containers vs. registries	Lecture
0:30 – 0:55	Anatomy of a Dockerfile; layer caching	Lecture + live demo
0:55 – 1:05	Break	—
1:05 – 1:30	Multi-stage builds	Lecture + live demo
1:30 – 1:50	GPU passthrough	Lecture
1:50 – 2:10	Case study: shared GPU lab machine contention	Discussion
2:10 – 2:55	Hands-on Exercise: naive vs. multi-stage image size comparison	Lab
2:55 – 3:00	Wrap-up: cheat sheet & Q&A	Plenary

Before You Start This Session

- Docker Desktop (Mac/Windows) or Docker Engine (Linux) installed and running — verify with `docker run hello-world`.
- A terminal and a code editor. GPU passthrough (Section 1.5) is conceptual unless your machine has an NVIDIA GPU and the NVIDIA Container Toolkit installed.
- Completion of Module 1 (reproducibility protocol: Git, Conda/Mamba, DVC, MLflow) is assumed — this lecture explicitly extends that protocol.
- This deck is self-contained: every concept is followed by a runnable code snippet, and the hands-on exercise (amber slide) references the companion Jupyter notebook.

00

Why Containers, After Everything We've Built?

What this section covers

- The one weak link in Module 1's reproducibility protocol
- What environment.yml can't pin
- What containers package instead

Environment.yml Pins Packages, Not the Machine

Code

pinned by Git

Data

pinned by DVC

OS / Drivers

NOT pinned by Conda/Mamba — the gap this
lecture closes

Discussion prompt: “You perfectly followed Module 1's protocol — pinned Git commit, pinned Conda environment, pinned DVC data. Your teammate still can't reproduce your result. What's left that ISN'T pinned?”

OS, System Libraries, GPU Drivers, Env Vars

- Two machines with an IDENTICAL environment.yml can still behave differently if their underlying OS or GPU drivers differ.
- System libraries (glibc, libgomp), CUDA/cuDNN versions, and environment variables all live OUTSIDE the Python process Conda manages.
- Containers solve this by packaging the ENTIRE runtime environment — OS libraries included — into a single, portable unit.

01

Images, Containers & Registries

What this section covers

- The core vocabulary of this entire lecture
- An analogy from object-oriented programming

Three Terms You'll Use All Lecture

1

Image

A read-only, layered template: a filesystem snapshot plus metadata. Built once from a Dockerfile.

2

Container

A running (or stopped) INSTANCE of an image, with its own writable layer on top.

3

Registry

Where images are stored and shared — Docker Hub, a private registry, or a cloud provider's registry.

1.2 A FAMILIAR ANALOGY

Image : Container :: Class : Object

- An image is a TEMPLATE — like a class definition in object-oriented programming.
- A container is an INSTANCE of that template — like an object created from a class.
- You can run MANY containers from the same one image, just as you can create many objects from one class.

02

Anatomy of a Dockerfile

What this section covers

- Instructions, layers, and the build cache
- Why instruction ORDER matters

Every Instruction Produces One Cached Layer

```
FROM python:3.10-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
CMD ["python", "train.py"]
```

Each instruction (FROM, COPY, RUN, ...) produces one layer. Layers stack to form the final image.

Why requirements.txt Is Copied BEFORE the Rest of the Code

- Docker caches each layer and reuses it on the next build if nothing that layer depends on has changed.
- Application code changes far more often than dependencies — so the expensive pip install step should live in an earlier, more stable layer.
- Result: most rebuilds skip dependency installation entirely and reuse the cached layer, saving minutes per build.

Watch the Cache in Action

```
$ docker build -t my-image .  
# ... first build: every layer runs ...  
  
# Change only a COMMENT in train.py (not requirements.txt)  
$ docker build -t my-image .  
# ... pip install layer says "CACHED" -- nearly instant rebuild ...  
  
# Now change requirements.txt itself  
$ docker build -t my-image .  
# ... pip install layer AND everything after it re-runs ...
```

Live demo: run this sequence on the projector before the lab — seeing "CACHED" in the terminal output is the single best way to build intuition for layer ordering.

03

Multi-Stage Builds

What this section covers

- Why naive AI images balloon to several GB
- Build stage vs. runtime stage

3.1 THE PROBLEM WITH NAIVE BUILDS

Build Tools End Up in the Final Image

- A naive Dockerfile for an AI application often ends up several GB, because build-time tools (compilers, dev headers, full CUDA toolkits) get baked into the final image alongside the actual runtime.
- Multi-stage builds fix this: use one stage to BUILD, and a second, minimal stage that only COPIES the finished artifacts across.

Two Stages, One Minimal Final Image

```
# --- Stage 1: build ---  
FROM python:3.10 AS builder  
WORKDIR /build  
COPY requirements.txt .  
RUN pip install --no-cache-dir --target=/build/deps -r requirements.txt  
  
# --- Stage 2: minimal runtime ---  
FROM python:3.10-slim  
WORKDIR /app  
COPY --from=builder /build/deps /usr/local/lib/python3.10/site-packages  
COPY . .  
CMD ["python", "serve.py"]
```

The final image contains only the Python runtime and installed packages — no compilers, no build caches, no dev headers.

Smaller Images, Smaller Everything

- Reduced image size directly reduces: pull time, attack surface, AND cold-start latency.
- Cold-start latency becomes critical in Lecture 2's KServe discussion — a scale-to-zero inference service pays this cost on every first request after idling.
- Rule of thumb worth internalizing now: everything you optimize in a Dockerfile today pays off again in Lecture 2.

04

GPU Passthrough

What this section covers

- Bridging container isolation and host hardware
- The NVIDIA Container Toolkit
- CUDA base image compatibility

Building FROM an NVIDIA Base Image

```
FROM nvidia/cuda:12.1.0-runtime-ubuntu22.04
```

```
RUN apt-get update && apt-get install -y python3 python3-pip
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
CMD ["python3", "train.py"]
```

The --gpus Flag

```
# Grant access to ALL host GPUs
$ docker run --gpus all my-training-image

# Restrict to a SPECIFIC GPU (useful on a shared multi-GPU machine)
$ docker run --gpus "device=0" my-training-image
```

Requires the NVIDIA Container Toolkit installed on the HOST machine — by default, containers cannot see host GPUs at all.

CUDA Version Compatibility, Not Identity

- The CUDA version inside the container's base image must be COMPATIBLE with the host's installed NVIDIA driver — not IDENTICAL to it.
- NVIDIA drivers are backward compatible with older CUDA runtime versions within a supported range.
- A container demanding a NEWER CUDA version than the host driver supports fails at ``docker run --gpus``, not at build time — a confusing debugging experience if you don't know to look here first.

05

Case Study

What this section covers

- Optimizing GPU access on a shared university lab machine

Two Research Groups, One Shared 2-GPU Machine

1

1. The Problem

Both groups' containers use `--gpus all`, so both processes contend for the SAME GPUs and memory.

2

2. Diagnose

Out-of-memory errors and slowdowns for BOTH groups, even though each thinks they own the machine.

3

3. Container Fix

Assign explicit device indices: `--gpus "device=0"` vs. `"device=1"` — exclusive access per group.

4

4. The Limitation

Manual assignment doesn't scale past a few users, and provides no fair time-sharing — motivates Lecture 2's K8s scheduler.

Naive vs. Multi-Stage Image Size Comparison

1. Open the companion notebook: `Lecture1_Docker_Exercise.ipynb`.
2. Write a naive, single-stage Dockerfile for the provided training script and build it; record the image size.
3. Rewrite it as a multi-stage build and build it; record the new image size and the percentage reduction.
4. Add a `.dockerignore` file and confirm it excludes unnecessary files from the build context.
5. Run the resulting container, with and without a `--gpus` flag, and note what changes (conceptually, if no GPU hardware is available).

Deliverable: A documented naive-vs-multi-stage image size comparison, a working `.dockerignore`, and a clear explanation of what `--gpus` all would change on GPU-equipped hardware.

Docker Cheat Sheet

Task	Command / Code
Build an image	<code>docker build -t <name> .</code>
List images and sizes	<code>docker images</code>
Run a container	<code>docker run <image></code>
Run with GPU access	<code>docker run --gpus all <image></code>
Run with ONE specific GPU	<code>docker run --gpus "'device=0'" <image></code>
Copy from an earlier build stage	<code>COPY --from=<stage_name> <src> <dest></code>
Exclude files from the build context	<code>.dockerignore</code>

KEY TAKEAWAY

A container packages the **WHOLE machine, not just the packages:**

Module 1's protocol + Containers = the last mile of true reproducibility

Every Dockerfile pattern in this lecture — layer ordering, multi-stage builds, minimal base images — exists to make containers small, fast, and reliable at cluster scale.

Next: Lecture 2, Part A — Kubernetes for AI/ML Workloads.